

Question 1 | Correct Mark 1.00 out of 1.00**Problem Statement**

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int countZeros(int arr[], int low, int high) {
4      if (low > high)
5          return 0;
6
7      // If the last element is 1, no zeroes are present
8      if (arr[high] == 1)
9          return 0;
10
11     // If the first element is 0, all remaining elements are 0
12     if (arr[low] == 0)
13         return high - low + 1;
14
15     int mid = (low + high) / 2;
16
17     return countZeros(arr, mid + 1, high) +
18         countZeros(arr, low, mid);
19 }
20
21 int main() {
22     int m;
23     scanf("%d", &m);
24
25     int arr[m];
26
27     for (int i = 0; i < m; i++) {
28         scanf("%d", &arr[i]);
29     }
30
31     printf("%d", countZeros(arr, 0, m - 1));
32
33     return 0;
34 }
```

	Input	Expected	Got	
✓	5 1 1 1 0 0	2	2	✓

	Input	Expected	Got	
✓	10 1 1 1 1 1 1 1 1 1 1 1 1	0	0	✓
✓	8 0 0 0 0 0 0 0 0 0 0	8	8	✓
✓	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`

Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-231 <= nums[i] <= 231 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int main() {
4      int n;
5      scanf("%d", &n);
6
7      int nums[n];
8
9      for (int i = 0; i < n; i++) {
10         scanf("%d", &nums[i]);
11     }
12
13     int candidate = nums[0];
14     int count = 1;
15
16     for (int i = 1; i < n; i++) {
17         if (nums[i] == candidate) {
18             count++;
19         } else {
20             count--;
21         }
22
23         if (count == 0) {
24             candidate = nums[i];
25             count = 1;
26         }
27     }
28 }
```

```
27     }
28
29     printf("%d", candidate);
30
31     return 0;
32 }
```

	Input	Expected	Got	
✓	3	3	3	✓
	3 2 3			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)



Question 1 | Correct Mark 1.00 out of 1.00**Problem Statement:**

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int findFloor(int arr[], int low, int high, int x) {
4      if (low > high)
5          return -1;
6
7      // If x is greater than or equal to the largest element
8      if (x >= arr[high])
9          return arr[high];
10
11     // If x is smaller than the smallest element
12     if (x < arr[low])
13         return -1;
14
15     int mid = (low + high) / 2;
16
17     // Exact match
18     if (arr[mid] == x)
19         return arr[mid];
20
21     // x lies between arr[mid] and arr[mid+1]
22     if (mid < high && arr[mid] < x && x < arr[mid + 1])
23         return arr[mid];
24
25     // Search left half
26     if (x < arr[mid])
27         return findFloor(arr, low, mid - 1, x);
28
29     // Search right half
30     return findFloor(arr, mid + 1, high, x);
31 }
32
33 int main() {
34     int n, x;
35
36     scanf("%d", &n);
37
38     int arr[n];
39
40     for (int i = 0; i < n; i++)
41         scanf("%d", &arr[i]);
42
43     scanf("%d", &x);
44
45     printf("%d", findFloor(arr, 0, n - 1, x));
46
47     return 0;
48 }

```

	Input	Expected	Got	
✓	6 1 2 8 10 12 19 5	2	2	✓
✓	5 10 22 85 108 129 100	85	85	✓
✓	7 3 5 7 9 11 13 15 10	9	9	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)



Question 1 | Correct Mark 1.00 out of 1.00**Problem Statement:**

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int findPair(int arr[], int left, int right, int x) {
4      if (left >= right)
5          return 0;
6
7      int sum = arr[left] + arr[right];
8
9      if (sum == x) {
10         printf("%d\n", arr[left]);
11         printf("%d\n", arr[right]);
12         return 1;
13     }
14
15     if (sum < x)
16         return findPair(arr, left + 1, right, x);
17     else
18         return findPair(arr, left, right - 1, x);
19 }
20
21 int main() {
22     int n, x;
23
24     scanf("%d", &n);
25
26     int arr[n];
27
28     for (int i = 0; i < n; i++) {
29         scanf("%d", &arr[i]);
30     }
31
32     scanf("%d", &x);
33
34     if (!findPair(arr, 0, n - 1, x))
35         printf("No");
36
37     return 0;
38 }
```

	Input	Expected	Got	
✓	4	4	4	✓
	2	10	10	
	4			
	8			
	10			
	14			
✓	5	No	No	✓
	2			
	4			
	6			
	8			
	10			
	100			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```

1  #include <stdio.h>
2
3  void quickSort(int arr[], int low, int high) {
4      if (low < high) {
5          int pivot = arr[high];
6          int i = low - 1;
7          int temp;
8
9          for (int j = low; j < high; j++) {
10             if (arr[j] <= pivot) {
11                 i++;
12
13                 temp = arr[i];
14                 arr[i] = arr[j];
15                 arr[j] = temp;
16             }
17         }
18
19         temp = arr[i + 1];
20         arr[i + 1] = arr[high];
21         arr[high] = temp;
22
23         int p = i + 1;
24
25         quickSort(arr, low, p - 1);
26         quickSort(arr, p + 1, high);
27     }
28 }
29
30 int main() {
31     int n;
32
33     scanf("%d", &n);
34
35     int arr[n];
36
37     for (int i = 0; i < n; i++) {
38         scanf("%d", &arr[i]);
39     }
40
41     quickSort(arr, 0, n - 1);
42
43     for (int i = 0; i < n; i++) {
44         printf("%d ", arr[i]);
45     }
46

```

```
47 |     return 0;  
48 | }
```

	Input	Expected	Got	
✓	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

